CHAPTER 1
INTRODUCTION

This paper describes and evaluates a teleoperated rescue robot built from commodity single-board hardware. The robot has two jobs. It must remove motor drive within a bounded time whenever its operator link or its own software fails, and it must let the operator see and talk to a trapped person. It is a four-wheel-drive ground vehicle: an Arduino UNO runs the motor firmware, and a Raspberry Pi 4 serves a browser dashboard over a local Wi-Fi network that has no Internet uplink. Section 1.1 gives the motivation, Section 1.2 states the four challenges (C1 to C4) that organise the rest of the paper, and Section 1.3 states the research questions, the contributions and the limits of the claims.

1.1 Motivation

In 2025 the Emergency Events Database recorded 358 natural-hazard disasters, which caused 16,607 deaths, affected 110.2 million people and cost USD 169.68 billion [**?**]. After such events rapid search and rescue is critical to survival, and the communications infrastructure that rescue teams rely on cannot be assumed to work [**?**]. Robots have been used in disaster response since the collapse of the World Trade Center in 2001, the first known use of robots for urban search and rescue [**?**]. In actual disasters rescue robots are typically teleoperated [**?**], and teleoperation is expected to remain an important default mode even for autonomous systems [**?**]. This paper therefore keeps the operator in the control loop and treats the link to the operator as something that will fail. The Fukushima Daiichi deployment shows why: tests indicated that robust wireless communication would not be possible inside the reactor buildings, so the robot sent there was fitted with a wired communication device [**?**]. A robot driven over a radio link has to decide for itself what to do when commands stop arriving.

A second need appears once the robot reaches a trapped person. A search-and-rescue (SAR) robot should provide audio-visual communication between the operator and trapped persons, so that an emergency responder can make a basic diagnosis [**?**]. On average four to ten hours pass between the discovery of a victim and extrication, during which the robot must keep the victim calm [**?**]. Two-way voice is already a fundamental function of most of the mine-rescue robots in a recent review [**?**]; what none of the recent low-cost platforms compared in Section 2.6 reports is two-way audio and video with the victim, or how much of a small computer that channel consumes while the same computer also drives and supervises the robot (Section 2.4).

1.2 Problem Statement and Challenges

The design problem is to build a teleoperated rescue robot on hardware cheap enough to be replaced, in which the safety behaviour does not depend on the weakest part of that hardware. Four challenges follow. Sections 2, 6 and 8 use the same labels and the same order.

C1: a bounded stop time on a non-real-time software stack. A stop decision taken by a Linux process inherits the scheduling and crash behaviour of that process (Section 2.3). If that process stalls or dies while the robot is driving, the last drive command stays in force. Link-loss failsafes and watchdog timers are established techniques [**?**, **?**]. The question for a low-cost rescue robot is where the decision runs and what time it achieves. Section 3.5 answers it with a dead-man timer in the motor-controller firmware.

C2: single points of failure in a single-board deployment. If control, video and supervision share one process on one board, a crash in any of them can end the mission. Recovery by restart, with several lines of defence because recovery can itself fail, is an established dependability strategy [**?**]. The challenge is to keep the robot either controllable or safely stopped when any one process fails, and to restore service without anyone going to the robot. Sections 3.4 and 3.7 answer it with a three-process layout and a watchdog process.

C3: two-way victim audio and video under CPU, power and bandwidth limits. Capturing, encoding and decoding audio and video competes with the control path for processor time, supply current and the one Wi-Fi cell. The challenge is to carry the victim channel on the same board without weakening the stop path, and to report what it costs. Section 3.9 describes the path and Section 5.5 reports the processor cost.

C4: affordability as a deployment constraint. Recent platforms are often described as low-cost, but none of those compared in Section 2.2 states a build cost. Here the bill of materials is reported as an estimate with its assumptions (Section 5.9), and no cost ratio against other platforms is claimed.

Each of these has prior work taken alone. What the platforms compared in Table 2.1 do not report together is a stop time bounded independently of the higher-level software, supervised recovery of the robot's processes, and two-way audio and video with the victim together with its resource cost, on low-cost hardware. This is a combination and not a first for any one part; Section 2.6 sets out the comparison and its limits.

1.3 Research Questions and Contributions

Four research questions follow from the challenges. Each is answered in Section 8.1 from the evidence of Section 5.

- **RQ1.** Can firmware on the motor controller bound the time at which drive is removed after the last command, whatever the state of the Pi and the operator link? (C1)

- **RQ2.** What latency does the control path add inside the robot, and how far does the link remain usable, with a single serial writer and a resumable session?
- **RQ3.** Does a watchdog restore service after a process is killed, without leaving the motors driven? (C2)
- **RQ4.** What does the two-way victim audio and video path cost the board, and how precise is the GPS position that the operator's map relies on, when both run on the same single-board computer? (C3)

RQ4 covers both the victim path and GPS with offline mapping, as in Sections 3.0 and 5 and Table 8.1.
The contributions, each evaluated in Section 5 and summarised in Table 8.2, are:

1. **A firmware dead-man timer with a ramp stop, independent of the Pi (C1; Section 3.5).** Drive was removed within 2042 ms of the last command in 30 of 30 logged trials, mean 2022.5 ms, against an analytical bound of about 2060 ms (MEASURED and CALCULATED; Section 5.1).
2. **A three-process split with a watchdog process and a single serial writer (C2; Sections 3.4, 3.6 and 3.7).** The watchdog restored service with a new process identifier in 30 of 30 SIGKILL trials on the bench, which gives a Wilson lower bound of 88.6% [**?**] (MEASURED; Section 5.4).
3. **A two-way audio and video path isolated in its own process (C3; Section 3.9).** One operator session used about 63% of one core on the media process while the control process stayed near 4% (MEASURED; Section 5.5).
4. **A commodity bill of materials (C4; Appendix A).** The itemised parts sum to approximately USD 310 from the authors' procurement receipts (37,158 BDT at an indicative 1 USD = 120 BDT, October 2026; Section 5.9).

The paper also states the evidence class of every figure in Section 5 and cross-checks hand-entered data against the robot's own machine logs (Sections 5 and 6.2).
The claims are limited, and Section 6.3 lists the limits in full. All results come from one robot, one operator and one access point, indoors or in an open area. The time reported in contribution 1 is the time at which drive is removed, not the time the wheels stop or a stopping distance; the first is not established and the second is not supported by the data (E3 and E4). Control latency over Wi-Fi, video and audio delay and quality, the end-to-end rescue run and the usability study were not measured and appear in Section 5 as [PENDING MEASUREMENT]. The ablations that would show what each mechanism contributes were not run. The stop behaves like a category 0 stop in IEC 60204-1 [**?**] but is not a safety-rated implementation, and no compliance is claimed.

The rest of the paper is organised as follows. Section 2 reviews related work and states the research gap. Section 3 describes the system, from the firmware stop upward. Section 4 defines the test conditions, metrics and baselines, and Section 5 reports the results. Section 6 interprets them and states their limits, Section 7 sets out future work, and Section 8 answers the research questions.